Iby and Aladar Fleischman
Faculty of Engineering
Tel Aviv University

הפקולטה להנדסה
ע"ש איבי ואלדר פליישמן
אוניברסיטת תל-אביב

The School of Electrical Engineering

ביי"ס להנדסת חשמל

Project Work Plan

Project Number: 3390

Project Name: A Reconfigurable, RISC-like Processing Element for Multi-Precision Neural Network Inference

Students:

- Nadav Dangur, 323132175
- Itai Schuster, 314777913

Project carried out at Tel-Aviv University.

Approval of the following is done by:

1. **Abstract**

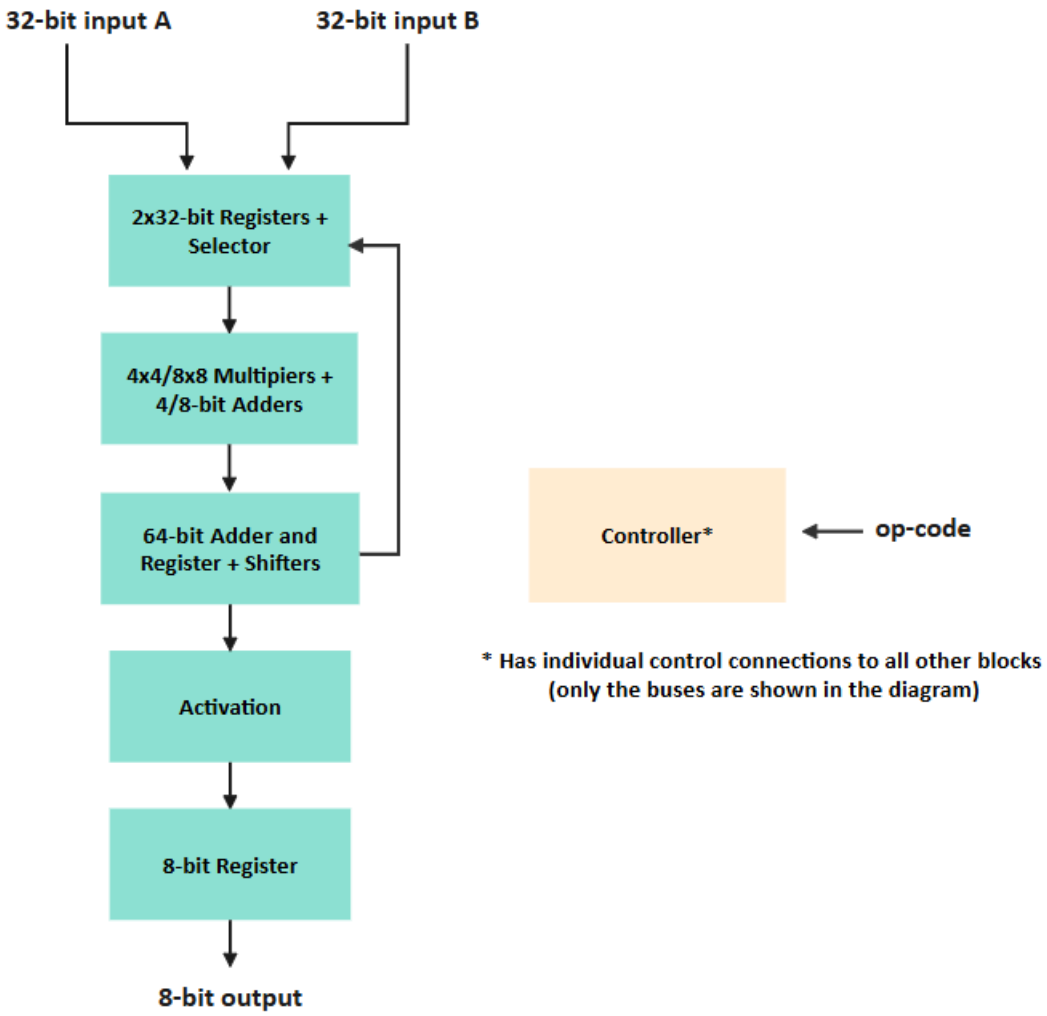
Subject
The project focuses on the design and implementation of a Quantized Processing Element (PE) - a small, dedicated hardware module that executes the core computational tasks of neural network inference efficiently and with minimal power consumption.

Field
This project belongs to the field of Digital Hardware Design, VLSI design and computer architecture.

Technological Applications
The technologies and methodologies used in this project are widely used in AI accelerators, edge devices, and system-on-chip (SoC) designs for deep learning inference.

Implementation Overview
The project follows a complete VLSI design flow, from architectural definition through RTL design, synthesis, and layout. The PE begins with a multiplication stage based on 8×8 units, which can also be reused as 4×4 multipliers to support multi-precision arithmetic with INT4 and INT8 operands. The resulting products are accumulated through accumulation logic, then passed to a rescale stage for normalization. Finally, an activation block applies non-linear functions, with alternative architectures evaluated to balance performance, power, and area. Key components, such as the multipliers, are shared across functional units to improve utilization and efficiency. Synthesis and physical implementation will employ Synopsys TSMC28 libraries. Collaboration with the FPGA Verification Platform project will ensure compatibility and joint verification across both ASIC and FPGA platforms.

Block Diagram



2. Motivation

Modern neural networks demand heavy computation, making it difficult for edge devices to stay powerful yet energy efficient. Imagine a smart toy that wakes up when it hears its name - most of the time it's idle, saving power, but instantly becomes active when needed. Our design follows this idea through three operation stages: dormant, where the hardware consumes almost no power; semi-dormant, where it listens and processes in low precision INT4; and fully active, where it switches to INT8 for high accuracy computation and begins executing the required commands. This adaptive precision allows the hardware to use just the resources it needs at each moment, reducing energy while keeping fast, accurate response.

Neural inference is commonly handled in two ways. The first uses powerful on-chip accelerators performing large scale parallel computations with formats like FP32 or FP16. While these deliver high throughput, they execute far more operations than needed for some small, event-driven edge tasks, wasting power, area, and bandwidth. Their fixed precision suits continuous workloads, not short, precise bursts of computation. The second approach relies on cloud-based inference, sending data to remote servers for processing. This provides abundant compute power but adds latency, communication overhead, and dependency on a stable network connection which is often impractical for portable or real-time systems. In contrast, our design allows low-precision INT4 operation during lightweight tasks, offering sufficient accuracy for specific applications while saving energy, and switching to INT8 only when full precision is required.

The proposed PE supports computation with small data formats such as INT8 and INT4, enabling precise yet highly efficient operation for low-power edge applications. Its design can be tailored to specific products, easily integrated or "stitched" into systems like voice-activated toys, wearable sensors, or monitoring devices that require compact and responsive inference hardware. Moreover, multiple PEs can be combined into a modular design, creating a scalable platform that can be tailored for specific applications or evolve into a shelf-ready product with configurable performance and power characteristics.

3. Work Statement

The project is based on VLSI design and digital hardware architecture, foundations of neural network inference, and the internal structure and operation of relevant hardware components (such as multipliers). We will also study Verilog and Synopsys tool flows, including the essential operating instructions needed for RTL development, synthesis, and analysis.

We will first define the PE architecture and micro-architecture and produce a custom instruction set & architectural checks document capturing op-code and the initial verification plan. In this phase we'll also lock the key decisions: INT4 mode (energy-saving reuse of 8×8s vs. high-throughput eight 4×4s), RESCALE (internal multi-cycle unit vs. staged RISC-like op-codes), and ACTIVATION (ROM/RAM LUT vs. synthesizable logic).

Each block (multipliers, shifters, controller, rescale, activation) will be implemented in Verilog with a matching unit testbench; simulations will run in VCS/ModelSim, and synthesis/P&R/STA will be done in Synopsys Fusion Compiler. For every block we'll collect coverage stats and retain the RTL, test files, and reports.

Verified blocks will be integrated into a synthesizable PE and exercised with a top-level regression (Python golden models + SystemVerilog benches in the same simulation environment), using RTL analysis where helpful; this yields the full-chip testbench and pass/fail summaries, coordinated with the FPGA team (validating our design on their platform and adjusting where needed).

Finally, we will complete layout, optimize, and report frequency, area, power, latency, and throughput. We'll highlight the critical path and explain the trade-offs behind the final design.

4. Deliverables

i. Architecture & Testing Documentation - architecture/micro-architecture spec, custom instruction set, architectural checks, and test plan.
ii. Design Choices Report - INT4/INT8 multiplier strategy, RESCALE path, and activation implementation.
iii. Verilog Project Description - complete RTL description of the PE.
iv. RTL Unit Testing Results & Coverage - per-block and full-PE results with coverage metrics.
v. Numerical Requirements Definition **-** target frequency, area, power, latency/throughput (frozen by the mid-project presentation).
vi. Verification Environment Testbenches - system-level and unit-level.
vii. Performance Documentation - power–performance–area analysis and latency–throughput trade-offs.
viii. PE Layout Files - final physical design files.
ix. Achievements & Goals Summary - wrap up of outcomes versus objectives.

5. Project Schedule

	Milestone	Description	Planned Date
1.	Architecture definition	Finish architecture definition. Make a document going over the tests needed for the PE (in coordination with FPGA team).	16.12.2025
2.	Micro-architecture definition	Finish designing the complete PE architecture. Reach definitive micro-architecture definition.	6.1.2026
3.	Implementation of 2x32 registers + selector block	Finish the RTL for the registers and selector (Verilog code). Write and perform unit tests on block.	29.1.2026
4.	Implementation of 4x4/8x8 multipliers and 4/8 bit-adders block	Finish the RTL for the multipliers and 4/8-bit adders block (Verilog code). Write and perform unit tests on block.	19.2.2026
5.	Implementation of the 64-bit adder and register + shifters block	Finish the RTL for the 64-bit adder, 64-bit register and shifters block (Verilog code). Write and perform unit tests on block.	2.3.2026
6.	Implementation of controller block	Finish the RTL for the controller block (Verilog code). Write and perform unit tests on block.	9.3.2026
7.	Implementation of ACTIVATE block	Finish the RTL for the activation function block (Verilog code). Write and perform unit tests on block.	16.3.2026
8.	Implementation of 8-bit register block	Finish the RTL for the 8-bit register block (Verilog code). Write and perform unit tests on block.	19.3.2026
9.	Progress presentation submission	Submit mid-project presentation. Decide on target PPA values as part of it.	25.3.2026
10.	Integration of blocks	Finish integration of blocks. Have a synthesizable RTL of PE (Verilog code).	8.4.2026
11.	Verification environment	Have a completely built verification environment for PE – SV testbenches + Python tests (in coordination with FPGA team).	15.4.2026

	Milestone	Description	Planned Date
12.	Verilog verification	Finish verification on Verilog code (in coordination with FPGA team).	1.5.2026
13.	Verification finish	Complete verification on PE (in coordination with FPGA team). Have the complete RTL ready to submit.	7.5.2026
14.	Synthesis + optimization	Finish the synthesis and optimization of the complete PE.	10.5.2026
15.	Layout	Have a physical layout design of the PE. PE is tapeout ready.	20.5.2026
16.	Poster submission and project finish	-	24.5.2026
17.	Project book submission and final presentation	-	July – August 2026